

ALF - Hoppings

Florian Goth

Universität Würzburg
Institut für Theoretische Physik und Astrophysik

16. Feb. 2022

- The hopping Hamiltonian
- Checkerboard Decomposition
- The most general entries of the hopping matrix
- Hubbard Model + The Operator type
- Currently defined structures

$$\hat{\mathcal{H}} = \hat{\mathcal{H}}_T + \hat{\mathcal{H}}_V + \hat{\mathcal{H}}_I + \hat{\mathcal{H}}_{0,I}$$

where

$$\hat{\mathcal{H}}_T = \sum_{k=1}^{M_T} \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger T_{xy}^{(ks)} \hat{c}_{y\sigma s} \equiv \sum_{k=1}^{M_T} \hat{T}^{(k)},$$

$$\hat{\mathcal{H}}_V = \sum_{k=1}^{M_V} U_k \left\{ \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger V_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \alpha_{ks} \right] \right\}^2 \equiv \sum_{k=1}^{M_V} U_k \left(\hat{V}^{(k)} \right)^2,$$

$$\hat{\mathcal{H}}_I = \sum_{k=1}^{M_I} \hat{Z}_k \left(\sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger I_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) \equiv \sum_{k=1}^{M_I} \hat{Z}_k \hat{I}^{(k)}.$$

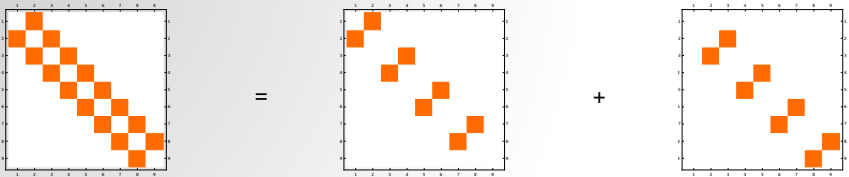
$$\hat{\mathcal{H}}_T = \sum_{k=1}^{M_T} \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger T_{xy}^{(ks)} \hat{c}_{y\sigma s} \equiv \sum_{k=1}^{M_T} \hat{\mathcal{T}}^{(k)}$$

- $T_{xy}^{(ks)}$, hopping matrix
- N_{dim} , spatial volume
- N_{fl} , number of flavours
- N_{col} , number of colours
- M_T , number of different hoppings (by model or due to checkerboard)
- We require only a single operation: $\exp(\Delta\tau \hat{\mathcal{H}}_T)$

We know the Lie-Trotter formula $e^{\Delta\tau(A+B)} = e^{\Delta\tau A} e^{\Delta\tau B} + \mathcal{O}(\Delta\tau^2)$.

Task: find a decomposition of $\hat{\mathcal{H}}_T$ as $T = \sum_{i=1}^{N_c} T^{(i)}$ such that for each $T^{(i)}$, $e^{T^{(i)}}$ can be efficiently evaluated.

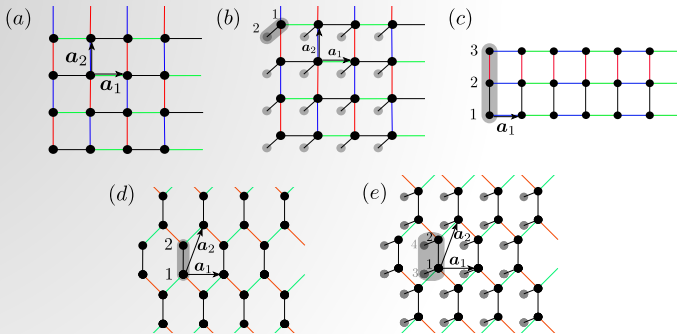
1D chain:



Each sub-matrix decomposes into blocks of the form

$$T^{(i)} = \begin{pmatrix} \ddots & & & \\ & 0 & c_j & \\ & c_j & 0 & \\ & & & \ddots \end{pmatrix} \rightarrow e^{T^{(i)}} = \begin{pmatrix} \ddots & & & \\ & \cosh(c_j) & \sinh(c_j) & \\ & \sinh(c_j) & \cosh(c_j) & \\ & & & \ddots \end{pmatrix}$$

Generalizations



Predefined Structures

→ Already worked out for common lattices!

The most generic hopping

$$\hat{H}_T = \sum_{(k,\delta),(j,\delta'),s,\sigma} T_{(k,\delta),(j,\delta'),s,\sigma}^{(s)} \hat{c}_{(k,\delta),s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{k+\delta}^{j+\delta'} \vec{A}^{(s)}(\vec{l}) d\vec{l}} \hat{c}_{(j,\delta'),s,\sigma}$$

with boundary conditions

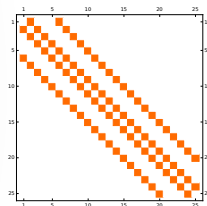
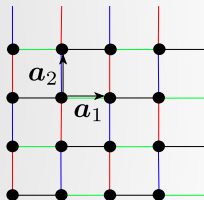
$$\hat{c}_{(k+L_i,\delta),s,\sigma}^\dagger = e^{-2\pi i \frac{\Phi_k^{(s)}}{\Phi_0}} e^{\frac{2\pi i}{\Phi_0} \chi_{L_k}^{(s)}(k+\delta)} \hat{c}_{(k,\delta),s,\sigma}^\dagger.$$

- k, j lattice indices
- δ, δ' orbital indices
- $\vec{A}^{(s)}(\vec{x}) = -B(x_2, 0, 0)$ the magnetic vector potential
- $T_{(k,\delta),(j,\delta'),s,\sigma}^{(s)}$ the non-magnetic hopping matrix
- $\Phi_k^{(s)}$ flux quantum
- $\chi_{L_k}^{(s)}(k+\delta)$ defined by $\vec{A}(\vec{x} + \vec{L}_i) = \vec{A}(\vec{x}) + \vec{\nabla} \chi_{L_i}(\vec{x})$.

Use Generic_hopping() for obtaining a matrix entry of this special form.

A worked example: Hubbard on square lattice

$$H = \sum_{\langle i,j \rangle, \sigma} c_{i\sigma}^\dagger T_{i,j} c_{j\sigma}$$



```
submodule (Hamiltonian_main) ham_my_hubbard_smod
Use Operator_mod
```

```
...
```

```
Implicit none
```

```
Type, extends(ham_base) :: ham_my_hubbard
```

```
contains
```

```
  procedure, nopass :: Ham_Set
```

```
  procedure, nopass :: Alloc_obs
```

```
  procedure, nopass :: Obser
```

```
  procedure, nopass :: ObserT
```

```
...
```

```
end type ham_my_hubbard
```

```
Type (Operator), dimension(:, :), allocatable :: Op_V ! -> base
```

```
Type (Operator), dimension(:, :), allocatable :: Op_T ! -> base
```

```
Integer :: Ndim ! -> base
```

```
Integer :: N_FL ! -> base
```

```
Integer :: N_SUN ! -> base
```

```
Integer :: Ltrot
```

```
Integer :: L1, L2
```

```
real (Kind=Kind(0.d0)) :: Ham_T, ham_U, Ham_chem
```

```
...
```


! Operator_mod.F90

Type Operator

...

complex :: O(:, :)

integer :: P(:)

integer :: N

complex :: g, alpha

...

End Type

User visible parts:

- $Op\%N$ effective dimension
- $Op\%O$ complex $N \times N$ matrix $T = P^T O P$
- $Op\%P$ a vector representation of the projector P .
- $Op\%g$ coupling strength $\Delta\tau T^{(k)}$
- $Op\%\alpha$ the constant α .

With this information we can recover the interaction matrix. $T = P^T O P$

For the product representation it holds

$$e^T = e^{P^T O P} = 1 + P^T (e^O - 1) P$$

Thereby we mostly have to exponentiate small matrices O .

Our convention: We have a Ham_Hop subroutine that sets up the hopping:

```
Subroutine Ham_Hop
  Implicit none
  Integer :: nf, I, Ix, Iy
  allocate(Op_T(1,N_FL))
  do nf = 1,N_FL
    Call Op_make(Op_T(1,nf),Ndim)
    Do I = 1,Latt%N
      Ix = Latt%nnlist(I,1,0)
      Op_T(1,nf)%O(I, Ix) = -Ham_T
      Op_T(1,nf)%O(Ix, I) = -Ham_T
      If ( L2 > 1 ) then
        Iy = Latt%nnlist(I,0,1)
        Op_T(1,nf)%O(I, Iy) = -Ham_T
        Op_T(1,nf)%O(Iy, I) = -Ham_T
      endif
      Op_T(1,nf)%O(I, I) = -Ham_chem
      Op_T(1,nf)%P(i) = i
    Enddo
    Op_T(1,nf)%g    = -Dtau
    Op_T(1,nf)%alpha = 0.d0
    Call Op_set(Op_T(1,nf))
  enddo
```

Then you can do *call Ham_Hop* in the subroutine Ham_set

hopping on a square lattice

$$\hat{H}_T = \sum_{i,\sigma,s} \left(\left[\sum_{\delta=\{\mathbf{a}_1,\mathbf{a}_2\}} -t^{(s)} \hat{c}_{i,s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+\delta} \vec{A}^{(s)}(\vec{l}) d\vec{l}} \hat{c}_{i+\delta,s,\sigma} + H.c. \right] - \mu^{(s)} \hat{c}_{i,s,\sigma}^\dagger \hat{c}_{i,s,\sigma} \right)$$

Call `Set_Default_hopping_parameters_square(Hopping_Matrix, T_vec, Chem_vec, Phi_X_vec, Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist, Latt, Latt_unit)`

Hopping on a honeycomb

$$\begin{aligned} \hat{H}_T = \sum_{i,\sigma,s} & \left(\sum_{\delta=\{\delta_1,\delta_2,\delta_3\}} -t^{(s)} \hat{c}_{i,s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+\delta} \vec{A}^{(s)}(\vec{l}) d\vec{l}} \hat{c}_{i+\delta,s,\sigma} + H.c. \right) \\ & + \sum_{i,\sigma,s} -\mu^{(s)} \left(\hat{c}_{i,s,\sigma}^\dagger \hat{c}_{i,s,\sigma} + \hat{c}_{i+\delta_1,s,\sigma}^\dagger \hat{c}_{i+\delta_1,s,\sigma} \right), \end{aligned}$$

Call `Set_Default_hopping_parameters_honeycomb(Hopping_Matrix, T_vec, Chem_vec, Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist, Latt, Latt_unit)`

See also in the docs

- Square/Honeycomb bilayer
- N-leg ladder

Using predefined hoppings

- Call `Set_Default_hopping_*`
- get `Hopping_Matrix_Type` object
- call `predefined_hoppings_set_OPT`

```
Type(Hopping_Matrix_Type), allocatable :: Hopping_Matrix(:)
...
call Set_Default_hopping_parameters_<prelattice>(Hopping_Matrix, ...
call Predefined_Hoppings_set_OPT(Hopping_Matrix, ..., Op_T)
```

Not covered here

- Ising Fields. See the chapter on Z_2 gauge theories in the doc.
- Internal structure of `Hopping_matrix_type`

Thank You for your Attention!